

```
// פונקציה עזר... מה מקבלת ומה מחזירה...
1 public static int length(Queue<Integer> q) {
2     Queue<Integer> tmp = new Queue<Integer>();
3     int count = 0;
4     while (!q.isEmpty()) {
5         tmp.insert(q.remove());
6         count++;
7     }
8     while (!tmp.isEmpty()) {
9         q.insert(tmp.remove());
10    }
11    return count;
12 }
13
14 Runtime complexity is  $O(n)$ 
```

```
// פונקציה עזר... מה מקבלת ומה מחזירה...
1 public static int atIndex(Queue<Integer> q, int indx) {
2     Queue<Integer> tmp = new Queue<Integer>();
3     while (indx > 0) {
4         tmp.insert(q.remove());
5         indx--;
6     }
7     int x = q.remove();
8     tmp.insert(x);
9     while (!q.isEmpty()) {
10        tmp.insert(q.remove());
11    }
12    while (!tmp.isEmpty()) {
13        q.insert(tmp.remove());
14    }
15    return x;
16 }
17 // Runtime complexity of atIndex() is  $O(n)$ 
```

"איבר קסם" הוא איבר בתור של מספרים שערכו שווה לסכום הערכים של האיבר שלפניו והאיבר שאחריו.
הערה: המספר הראשון בתור והמספר האחרון בתור אינם "איברי קסם".

א. כתבו פונקציה ששמה isMagic בשפת Java או IsMagic בשפת C#, המקבלת תור – q – מטיפוס שלם, ומספר שלם – m – הגדול מ-0 וקטן או שווה לגודל התור. הפעולה תחזיר true אם האיבר במקום ה-m בתור הוא "איבר קסם", אחרת היא תחזיר false.

```
1 public static boolean isMagic(Queue<Integer> q, int m) {
2     if (m == 1 || m == length(q)) {
3         return false;
4     }
5
6     // לפי השאלה המיקומים בתור מתחילים באחד
7     int l = atIndex(q, m-2);
8     int x = atIndex(q, m-1);
9     int r = atIndex(q, m);
10    return x == l + r;
11 }
```

"איבר קסם" הוא איבר בתור של מספרים שערכו שווה לסכום הערכים של האיבר שלפניו והאיבר שאחריו.
הערה: המספר הראשון בתור והמספר האחרון בתור אינם "איברי קסם".

ב. כתבו פונקציה ששמה nMagic בשפת Java או NMagic בשפת C# המקבלת תור מטיפוס שלם – q, ומספר שלם – n הגדול מ-0 וקטן או שווה לגודל התור. הפעולה תחזיר true אם כל האיברים הנמצאים במקומות שהם כפולה של n (המקום ה-n בתור, המקום ה-2n בתור וכן הלאה בדילוגים של n מקומות) הם "איברי קסם", אחרת הפעולה תחזיר false.

```
1 public static boolean nMagic(Queue<Integer> q, int n) {
2     while (n <= length(q)) {
3         if (!isMagic(q, n)) {
4             return false;
5         }
6         n += n;
7     }
8     return true;
9 }
```

```

1 // insert number into ordered (ascending) queue
2 public static void insertOrdered(Queue<Integer> q, int num)
3     Queue<Integer> tmp = new Queue<Integer>();
4     while (!q.isEmpty() && q.head() <= num) {
5         tmp.insert(q.remove());
6     }
7     tmp.insert(num);          // מנסה להכניס את המספר החדש לזרם
8     while (!q.isEmpty()) {
9         tmp.insert(q.remove());
10    }
11    while (!tmp.isEmpty()) {
12        q.insert(tmp.remove());
13    }
14 }

```

```

1 class Patient {
2     private int id;
3     private int priority;
4
5     public int getPriority() { return this.priority; }
6 }
7
8 //... הפונקציה מקבלת... ומכניסה לתור
9 public static void insertWithPriority(Queue<Patient> q,
10                                     Patient p)
11 {
12     Queue<Patient> tmp = new Queue<Patient>();
13     while (!q.isEmpty() &&
14           q.head().getPriority() >= p.getPriority())
15     {
16         tmp.insert(q.remove());
17     }
18     tmp.insert(p);
19     while (!q.isEmpty()) {
20         tmp.insert(q.remove());
21     }
22     while (!tmp.isEmpty()) {
23         q.insert(tmp.remove());
24     }
25 }

```

```

1 // remove and return a person from a queue
2 public static Patient remove(Queue<Patient> q, int id) {
3     Queue<Patient> tmp = new Queue<Patient>();
4     while (!q.isEmpty() && q.head().getId() != id) {
5         tmp.insert(q.remove());
6     }
7     Patient p = q.remove();
8     while (!q.isEmpty()) {
9         tmp.insert(q.remove());
10    }
11    while (!tmp.isEmpty()) {
12        q.insert(tmp.remove());
13    }
14    return p;
15 }

```

```

1 public class PriorQueue {
2     private Queue<Patient> q;
3
4     public PriorQueue() {
5         this.q = new Queue<Patient>();
6     }
7
8     public void priorityInsert(Patient p) {
9         insertWithPriority(this.q, p);
10    }
11
12    public void update(int id, int pri) {
13        Patient p = remove(this.q, id);
14        p.setPriority(pri);
15        insertWithPriority(this.q, p);
16    }
17 }

```

lst	x	lst==null	temp	lst.getValue()==x
עקבו אחר הפעולה (lst, 1) [1,3,5,3,1,9,4] [3,5,3,1,9,4] [5,3,1,9,4] [3,1,9,4] [1,9,4] [9,4] [4] []	1	false false false false false false false true	[3,5,3,9,4] [3,5,3,9,4] [5,3,9,4] [3,9,4] [9,4] [9,4] [4] null	1==1: true 3==1: false 5==1: false 3==1: false 1==1: true 9==1: false 4==1: false

```

public static Node<Integer> what (Node<Integer>lst, int x)
{
    if (lst == null)
        return null;

    Node<Integer> temp = what (lst.getNext(),x);

    if (lst.getValue() == x)
        return temp;

    lst.setNext (temp);

    return lst;
}

```

עוברים על כל עברים של רשימה
בירידה ברקורסיה ואז עוד פעם בחזרה
לכן סיבוכיות זמן ריצה היא $O(n)$

מפרקת את ראש הרשימה בירידה ברקורסיה
מחזירה את הראש עם הוא לא שווה ל-x בחזרה
בזה מוחקת את ה-x מהרשימה $O(n)$

guess(lst)	lst!=null	what([],1)	temp	global lst
עקבו אחר הפעולה (lst) הגאנץ' [1,3,5,3,1,9,4] [3,5,3,9,4] [5,9,4] [4] []	false false false false true	[3,5,3,1,9,4],1 [5,3,9,4],3 [9,4],5 [],9	[3,5,3,9,4] [5,9,4] [9,4] [9]	[1,3,5,3,9,4] [1,3,5,9,4] [1,3,5,9,4] [1,3,5,9,4]

```

public static void guess (Node<Integer>lst)
{
    if (lst != null) {
        Node<Integer> temp = what (lst.getNext(), lst.getValue());

        lst.setNext (temp);

        guess (lst.getNext());
    }
}

```

פעולה guess מוחקת מהרשימה את הכפילויות
זה נעשה בעזרת פונקציה what

עבור כל איבר ברשימה אנו עוברים
על רשימה עוד פעם (ללא ראש)
סיבוכיות זמן ריצה היא $O(n^2)$

arr – מערך מטיפוס שלם בגודל 10 , המכיל את מספרי קווי האוטובוס שעוצרים בתחנה. בתחנה עוצרים ענן 10 קווים, והם נשמרים ברצף מתחילת המערך.

amount – כמות קווי האוטובוס שעוצרים בתחנה במהלך, מטיפוס שלם. בתחנה עוצר קו אוטובוס אחד לפחות.

```

1 class BusStation {
2     private int num;
3     private int[] arr;
4     private int amount;
5
6     public int[] getArr() { return this.arr; }
7     public int getAmount() { return this.amount; }
8

```

הפעולה מקבלת מספר קו אוטובוס – n מטיפוס שלם. הפעולה מחזירה true אם קו האוטובוס עוצר בתחנה, ואחרת מחזירה false.

```

9     public boolean isStopping(int n) {
10         for (int i = 0; i < this.amount; i++) {
11             if (arr[i] == n) {
12                 return true;
13             }
14         }
15         return false;
16     }
17 }

```

נתון מערך arr מטיפוס BusStation, ובו כל תחנות האוטובוס בעיר ממוימות. ידוע שכמה מקווי האוטובוס עוצרים בכל אחת מן התחנות. ושאר הקווים עוצרים רק בחלק מן התחנות.

כתבו פעולה חיצונית ששמה allStations בשפת Java או AllStations בשפת C#, המקבלת את המערך arr. הפעולה מחזירה שרשרת חוליות מטיפוס שלם שבכל חוליה מופיע אחד ממספרי הקווים שעוצרים בכל התחנות בעיר (כל קו כזה יופיע פעם אחת בלבד בשרשרת).

תענה: אין חשיבות לסדר הקווים בשרשרת.

```

1 var city = new BusStation[] {
2     new BusStation(1, new int[] {10, 20, 30}),
3     new BusStation(2, new int[] {10, 30, 40}),
4     new BusStation(3, new int[] {10, 20, 30}),
5     new BusStation(4, new int[] {10, 20, 30, 40}),
6 };
7
8 print(allStations(city)); // [30, 10]
9
10 public static boolean inCity(BusStation[] arr, int n) {
11     for (BusStation bs: arr) {
12         for (int i = 0; i < bs.getAmount(); i++) {
13             if (bs.getArr()[i] == n) {
14                 return true;
15             }
16         }
17     }
18     return false;
19 }

```

```

1 var city = new BusStation[] {
2     new BusStation(1, new int[] {10, 20, 30}),
3     new BusStation(2, new int[] {10, 30, 40}),
4     new BusStation(3, new int[] {10, 20, 30}),
5     new BusStation(4, new int[] {10, 20, 30, 40}),
6 };
7
8 // בודקת האם קו עוצר בכל התחנות בעיר
9 public static boolean stopsEverywhere(BusStation[] arr,
10                                         int n)
11 {
12     for (BusStation bs: arr) {
13         if (!bs.isStopping(n)) {
14             return false;
15         }
16     }
17     return true;
18 }
19
20 // בודקת האם איבר בתוך הרשימה
21 public static boolean contains(Node<Integer> lst, int n) {
22     while (lst != null) {
23         if (lst.getValue() == n) {
24             return true;
25         }
26         lst = lst.getNext();
27     }
28     return false;
29 }

```

```

1 var city = new BusStation[] {
2     new BusStation(1, new int[] {10, 20, 30}),
3     new BusStation(2, new int[] {10, 30, 40}),
4     new BusStation(3, new int[] {10, 20, 30}),
5     new BusStation(4, new int[] {10, 20, 30, 40}),
6 };
7
8 public static boolean contains(Node<Integer> lst, int n) {
9     ...
10 }
11
12 public static boolean stopsEverywhere(BusStation[] arr,
13                                         int n)
14 {
15     ...
16 }
17
18 public static Node<Integer> allStations(BusStation[] arr) {
19     Node<Integer> lst = null;
20     for (BusStation bs: arr) {
21         for (int i = 0; i < bs.getAmount(); i++) {
22             int n = bs.getArr()[i];
23             if (stopsEverywhere(arr, n) &&
24                 !contains(lst, n))
25             {
26                 lst = new Node<Integer>(n, lst);
27             }
28         }
29     }
30     return lst;
31 }

```

"תת-סדרה נגדית" היא רצף של מספרים המתחיל במספר כלשהו ומסתיים במספר הנגדי והשווה לו בערך המוחלט שלהם (כלומר מתחיל במספר חיובי ומסתיים במספר השלילי המקביל לו או מתחיל במספר שלילי ומסתיים במספר החיובי המקביל לו). הרצף נמצא בתוך סדרה של מספרים.

לדוגמה: בסדרת המספרים: 17, 3, 1, -4, 5, 6, 4, -3, 7, 23, -99 יש שתי תת-סדרות נגדיות:

i. 3, 1, -4, 5, 6, 4, -3
 ii. -4, 5, 6, 4

א. כתבו פונקציה ששמה width בשפת Java או בשפת C# המקבלת את השרשרת lst ומספר שלם num (חיובי או שלילי) המופיע בשרשרת. הפונקציה תחזיר את אָנֹכֶבּ ה"תת-סדרה נגדית" שהמספר num מתחיל או מסיים (האורך כולל את המספרים בקצוות). אם המספר הנגדי ל־num אינו מופיע בשרשרת, הפונקציה תחזיר -1.

```

1 public static int width(Node<Integer> lst, int num) {
2     return Math.max(
3         width(lst, -num, num),
4         width(lst, num, -num));
5 }

```

ב. כתבו פונקציה ששמה longest בשפת Java או בשפת Longest בשפת C# המקבלת את השרשרת lst. הפונקציה תחזיר את אָנֹכֶבּ ה"תת-סדרה נגדית" הגדולה ביותר. אם אין בשרשרת שום "תת-סדרה נגדית", הפונקציה תחזיר -1. אפשר להשתמש בפונקציה שכתבתם בסעיף א.

```

1 public static int longest(Node<Integer> lst) {
2     Node<Integer> node = lst;
3     int max = -1;
4     while (node != null) {
5         max = Math.max(max, width(lst, node.getValue()));
6         node = node.getNext();
7     }
8     return max;
9 }

```

א. כתבו פונקציה ששמה width בשפת Java או בשפת C# המקבלת את השרשרת lst ומספר שלם num (חיובי או שלילי) המופיע בשרשרת. הפונקציה תחזיר את אָנֹכֶבּ ה"תת-סדרה נגדית" שהמספר num מתחיל או מסיים (האורך כולל את המספרים בקצוות). אם המספר הנגדי ל־num אינו מופיע בשרשרת, הפונקציה תחזיר -1.

```

1 public static int width(Node<Integer> lst, int left,
2                             int right)
3 {
4     int first = -1;
5     int last = -1;
6     int i = 0;
7     while (lst != null) {
8         if (lst.getValue() == left && first == -1) {
9             first = i;
10        }
11        if (lst.getValue() == right) {
12            last = i;
13        }
14        lst = lst.getNext();
15        i++;
16    }
17
18    if (first == -1 || last == -1 || first > last) {
19        return -1;
20    }
21    return last - first + 1;
22 }

```


אם L_1 ו- L_2 אינן רגולריות
אז $L_1 \cap L_2$ בהכרח אינה רגולרית
לא נכון
 $L_1 = \{a^n b^n\}$
 $L_2 = \overline{L_1}$
 $L_1 \cap L_2 = \emptyset$

$L_1^n \cdot L_2^n \Leftrightarrow (L_1 \cdot L_2)^n$
 $L^n = \underbrace{L \cdot L \cdots L}_{n \text{ times}}$

לא נכון
 $L_1 = \{a\} \Rightarrow L_1^2 = \{aa\}$
 $L_1 = \{b\} \Rightarrow L_1^2 = \{bb\}$
 $L_1 \cdot L_2 = \{ab\}$
 $L_1^2 \cdot L_2^2 = \{aabb\}$
 $(L_1 \cdot L_2)^2 = \{abab\}$

$L = \{w \mid w \text{ אינם מופיעים ב- } ab, bc, \#_a(w) + \#_c(w) = \#_b(w)\}$

cba	כן	ϵ	כן
cbbba	כן	aab	לא
baa	כן	aaccc	לא